DOMAIN 2 // RECONNAISSANCE & ENUMERATION

COMPTIA PENTEST+ PT0-003 · OBJECTIVES V3.0 · FULL REFERENCE

21% OF EXAM · ~19 QUESTIONS

THE ORGANIZING AXIS — READ THIS FIRST

- **Passive** — no packets reach infrastructure the target owns. You query registrars, public logs, search-engine indexes, breach corpora and archives. The client's SOC has nothing to detect.
  EXPECT "without alerting the client", "before the testing window opens", "generates no traffic to the target" → the answer is a passive technique or tool.
- **Active** — the target receives and logs your traffic. Port scanning, banner grabbing, crawling their site, zone-transfer attempts, frame injection.
  EXPECT Active options are eliminated the moment a question adds a timing or stealth constraint. Check that first; it often halves the option list.

THE CLASSIC PASSIVE/ACTIVE PAIR

A standard DNS lookup goes to a public recursive resolver — the target never sees it, so it is *passive*. An AXFR zone-transfer request goes straight to the target's authoritative name server, which receives, logs and usually refuses it — *active*. Same protocol, opposite sides of the line.

2.1 INFORMATION GATHERING — GIVEN A SCENARIO, APPLY

OSINT SOURCES

- **Social media** — names, titles, reporting lines, tech mentions, badge and interior photos.
  EXPECT Feeds spearphishing pretexts and physical-assessment planning, not technical exploitation.
- **Job boards** — the best passive source for the technology stack, often with version numbers.
  EXPECT "Determine what technology they run, from public sources" → job boards. Also reveals team size and gaps.
- **Scan code repositories** — hardcoded credentials, API keys, internal hostnames, connection strings.
  EXPECT Secrets deleted from current files remain in commit history. Remediation = rotate first, then secrets management (1.5).
- **DNS lookups / reverse lookups** — forward = name→IP; reverse = IP→name via PTR. Records: A, AAAA, MX, NS, TXT, CNAME, SOA.
  EXPECT The value of a reverse sweep is the *naming convention* it exposes, letting you predict hosts you never observed.
- **Cached pages** — Google cache and the Wayback Machine hold content the org has removed.
  EXPECT "Removed", "took down", "no longer published" → cached pages / Wayback Machine.
- **Cryptographic flaws** — deprecated SSL/TLS, weak ciphers and key exchange, expired or self-signed certs, named issues such as DROWN.
  EXPECT Certificate *contents* are intelligence too — SANs routinely disclose internal hostnames and extra domains.
- **Password dumps** — valid usernames, the org's email format, real password patterns, reuse.
  EXPECT "Valid usernames and likely password patterns" → password dumps. Feeds spraying and stuffing in 4.3.

NETWORK-LEVEL GATHERING

- **Network reconnaissance** — netblock ownership via regional registries, ASN and BGP data, traceroute path and edge devices, CDN detection.
  EXPECT Doubles as scope validation from 1.1 — confirming an address belongs to the client and not a hosting provider.
- **Protocol scanning (TCP/UDP)** — TCP connect completes the handshake and is logged by the application; SYN (half-open) is faster and quieter but needs raw-socket privileges; UDP is slow and ambiguous.
  EXPECT UDP results reported as `open|filtered` mean *no response was received*. A closed UDP port returns ICMP type 3 code 3.

RESPONSE	STATE
SYN/ACK	Open
RST	Closed
No response / ICMP unreachable	Filtered
No response on UDP	Open *or* filtered — ambiguous

OTHER 2.1 TECHNIQUES

- **Certificate transparency logs** — public append-only records of every issued certificate. Surfaces `dev.`, `staging.`, `vpn.` hosts no wordlist would produce.
  EXPECT "Enumerate subdomains without sending traffic to the target" → CT logs. Highest-yield passive answer in 2.1.
- **Information disclosure** — verbose errors with stack traces and paths, `Server` / `X-Powered-By` headers, directory listings, EXIF and document metadata, source comments, `robots.txt`, exposed `.git`.
  EXPECT The discriminator vs OSINT: OSINT is the discipline of collecting public data; information disclosure is what the org leaked *by accident*.
- **Search engine analysis** — dorking with `site:` `filetype:` `inurl:` `intitle:` `intext:` and `-site:` to exclude the main domain.
  EXPECT Google indexes *content*; Shodan and Censys index *services and banners*. Match the index to the question.
- **Network sniffing** — cleartext credentials, hostnames, VLAN tags, LLMNR/NBT-NS/mDNS chatter, CDP/LLDP device details. Requires segment access or a mirrored port.
  EXPECT The IoT/OT sub-bullet: Modbus (502), DNP3, S7comm, BACnet, EtherNet/IP, MQTT have *no authentication or encryption by design*, so capture yields register values directly.
- **Banner grabbing** — connect and read the greeting (netcat, telnet, `curl -I`, `nmap -sV`). Active by definition.
  EXPECT Gives the *service version*. A missing or generic banner is itself a signal that the host was hardened.
- **HTML scraping** — extracting emails, names, framework fingerprints, comments, hidden fields, API endpoints from pages you already have.
  EXPECT Scraping *extracts from* a page; crawling *discovers* pages by following links — and crawling is 2.2, not 2.1.

2.1 DIFFERENTIATORS

Banner grabbing vs OS fingerprinting — service version from a volunteered greeting vs operating system inferred from stack behaviour (TTL, window size, option ordering).
DNS lookup vs zone transfer — public resolver (passive) vs the target's authoritative name server (active).
Forward vs reverse DNS — name to address vs address to name via PTR.
CT logs vs cryptographic flaws — discovering hostnames from issued certificates vs assessing weak crypto configuration.
Network reconnaissance vs protocol scanning — who owns the address space vs which ports respond.

2.2 ENUMERATION GIVEN A SCENARIO, APPLY

Enumeration is layered. Almost every "which technique" question resolves by identifying the layer: **host discovery** (is anything alive) → **service discovery** (what is listening) → **protocol enumeration** (what will it tell me) → **identity enumeration** (who, and what rights) → **attack path mapping** (how do I chain it).

LAYERS 1–3 — REACHING THE HOST

- **Host discovery** — ARP scan (most reliable locally, cannot be filtered without breaking the network), ICMP echo, TCP SYN/ACK ping, UDP ping.
  EXPECT No ping reply does not mean the host is down — it is a statement about their firewall. Skip discovery with `nmap -Pn`.
- **Service discovery** — which ports are open and what software is bound. `nmap -sV` probes and matches against a signature database.
  EXPECT It identifies software that publishes *no banner at all*, which is precisely how it differs from banner grabbing in 2.1.
- **OS fingerprinting** — initial TTL (64 Linux, 128 Windows, 255 network gear), TCP window size, option ordering, reaction to malformed packets. Active `nmap -O` or passive from captured traffic.
  EXPECT A banner names a *service*; fingerprinting names an *operating system*. The exam pairs them constantly.
- **Protocol enumeration** — interrogating the protocol for data, not merely detecting it.
  EXPECT SNMP with a default community string returns interfaces, routes, ARP caches, processes, installed software. SMTP `VRFY` / `EXPN` / `RCPT TO` confirms mailboxes. LDAP returns users, groups, OUs, password policy. NFS `showmount -e` lists exports.
- **DNS enumeration** — subdomain brute forcing, zone-transfer attempts, wildcard detection, cache snooping, SRV queries.
  EXPECT `_ldap._tcp.dc._msdcs.<domain>` locates Active Directory domain controllers in one query, far quieter than scanning for them.
- **Wireless enumeration** — SSIDs including hidden, BSSIDs, channels, encryption, signal strength, connected clients.
  EXPECT A hidden SSID appears the moment a client associates, or after a deauth forces reassociation. Hiding is obscurity, not a control.

LAYER 4 — IDENTITY ENUMERATION

- **Share enumeration** — `smbclient -L`, `enum4linux`, `crackmapexec smb --shares`, `showmount -e`.
  EXPECT Shares hold credentials in scripts, backups and config files — one of the highest-yield activities in a real engagement.
- **Local user enumeration** — `net user`, `net localgroup`, RID cycling, SAM contents; on Linux `/etc/passwd`, `getent`, `id`, `last`.
  EXPECT Against a *host or directory you can reach* — that is the split from email account enumeration.
- **Email account enumeration** — SMTP verb responses, differential webmail or password-reset replies, harvesting, deriving the format from one known address.
  EXPECT Against the org's *external identity surface*. Produces the phishing and spraying target lists.
- **Permission enumeration** — `whoami /priv`, `/groups`, AccessChk, service permissions, unquoted service paths; on Linux `sudo -l`, SUID/SGID, writable cron; in cloud, effective IAM evaluation.
  EXPECT This is the layer that produces the privilege-escalation route. Under-studied relative to how often it appears.
- **Secrets enumeration** — four named categories: *cloud access keys* (repos, env vars, `~/.aws/credentials`, instance metadata), *passwords* (config files, unattend files, GPP, browser stores, memory), *API keys* (source, JS bundles, mobile packages), *session tokens* (cookies, JWTs, Kerberos tickets, OAuth).
  EXPECT The bridge into Domain 5. Any find here carries the 1.5 remediation: rotate, then secrets management.

LAYER 5 AND THE WEB TRACK

- **Attack path mapping** — modelling collected data as a graph from current position to objective. BloodHound: nodes are users/groups/computers, edges are rights such as `MemberOf`, `AdminTo`, `GenericAll`.
  EXPECT It *consumes* enumeration output; it collects nothing new. Same insight as the 1.4 attack narrative — harmless rights chain into compromise.
- **Directory enumeration** — requesting candidate paths from a wordlist. Gobuster, DirBuster, Wfuzz, ffuf.
  EXPECT Finds admin panels, backups, `.git` and API endpoints that *nothing links to*.
- **Web crawling** — following existing links to map the application. Burp and ZAP spiders.
  EXPECT Yields pages, parameters, forms and JS files reachable through navigation — and nothing that is unlinked.
- **Manual enumeration** — `robots.txt` (paths the org asked search engines to skip — a curated list of what they consider sensitive), `sitemap.xml` (their own complete page inventory), platform plugins (CMS and plugin versions).
  EXPECT Plugins are where the vulnerabilities live, not the core CMS. WPScan handles this in 4.5.
- **WAF enumeration** — detection from headers, cookies, block pages and payload behaviour (wafw00f); then the *origin address*.
  EXPECT The sub-bullet is the tested part. Reach the origin via historical DNS, CT logs, non-CDN subdomains (`mail.`, `ftp.`, `dev.`), SSRF, or a Censys certificate search. If the origin accepts connections from anywhere, the WAF is void — remediation is restricting it to CDN ranges.

2.2 DIFFERENTIATORS — DRILL THESE

Host vs service discovery — is anything there vs what is listening.
Service discovery vs protocol enumeration — the protocol is present vs what the protocol will tell me if I ask.
Service discovery vs banner grabbing — active probing and fingerprinting vs reading a volunteered greeting.
Directory enumeration vs web crawling — guessing unlinked paths vs following existing links. *Guessing vs following.*
Local user vs email account enumeration — accounts on a reachable host vs the org's external identity surface.
Permission enumeration vs attack path mapping — what one identity can do vs how rights chain to the objective.
WAF enumeration vs origin address — the WAF exists and is product X vs the IP that bypasses it entirely.
DNS lookups (2.1) vs DNS enumeration (2.2) — resolving names you know vs discovering names you do not.

2.3 SCRIPTING GIVEN A SCENARIO, MODIFY SCRIPTS

The verb is **modify**, not write. You are shown a snippet and asked what it does, what is broken, or which line achieves a goal. CompTIA writes distractors that are valid syntax *in a different language* — so identify the language before reading any logic.

IDENTIFY THE LANGUAGE IN TWO SECONDS

MARKER	LANGUAGE
`#!/bin/bash`, `$(cmd)`, `; then`, `; do`, `fi`, `done`	Bash
`import`, `def name():`, **colon + indentation,** `print()`	Python
Verb-Noun cmdlets, `$_`, `-eq` **without brackets,** `1..254`, `New-Object`	PowerShell

- EXPECT `-eq` alone is not a discriminator: Bash uses it *inside* test brackets, PowerShell uses it bare. `$_` is uniquely PowerShell. Colon-plus-indentation is uniquely Python.

ANATOMY 1 — BASH PING SWEEP

```
01 #!/bin/bash
02 NET="10.0.0"
03 for i in $(seq 1 254); do
04   ping -c 1 -W 1 "$NET.$i" &>/dev/null && echo "$NET.$i up" >> live.txt &
05 done
06 wait
```

01 Shebang — tells the kernel which interpreter runs the file. Its absence is a planted defect when the script is executed directly.

02 Assignment. **No spaces around** `=` — `NET = "10.0.0"` is parsed as running the command `NET`. This is the opposite of Python and PowerShell.

03 `$( )` is command substitution — it runs `seq` and substitutes the output. `seq 1 254` is inclusive at both ends. `do` opens the body; `done` on line 05 closes it.

04 `-c 1` sends one packet; `-W 1` caps the wait at one second. `&>/dev/null` discards stdout *and* stderr. `&&` runs the right side only if the left exited 0. `>>` appends — a single `>` would truncate the file every iteration. The trailing `&` backgrounds each probe so they run in parallel.

05 `done` closes the `for`. A missing `done` / `fi` is a standard planted bug.

06 `wait` blocks until every backgrounded job finishes, so the script does not exit while probes are still running.

- EXPECT "The script takes hours to complete" → the missing `&` or the missing `-W`. "The output file only ever has one line" → `>` used where `>>` was needed.

ANATOMY 2 — PYTHON PORT SCANNER

```
01 import socket
02
03 target = "10.0.0.5"
04 open_ports = []
05
06 for port in range(1, 1025):
07     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
08     s.settimeout(0.5)
09     if s.connect_ex((target, port)) == 0:
10         open_ports.append(port)
11     s.close()
12
13 print(f"Open: {open_ports}")
```

01 `import` brings a module's namespace in. Match the library to the task: `socket` raw TCP, `requests` HTTP, `subprocess` shell out to another tool, `re` regex, `scapy` packet crafting, `impacket` SMB/Kerberos.

03–04 Assignment needs no type declaration. `[]` is an empty list literal used to accumulate results.

06 `range(start, stop)` — **stop is exclusive**, so this covers 1–1024. The colon opens the block and *indentation* defines membership; there are no braces.

07 `AF_INET` = IPv4, `SOCK_STREAM` = TCP. A UDP socket would be `SOCK_DGRAM`.

08 Timeout in seconds. Without it, filtered ports hang on the OS default — the standard explanation for a scanner that "freezes".

09 `connect_ex` returns an error code and **0 means success**. Its sibling `connect()` raises an exception instead, killing the loop at the first closed port. Note the double parentheses: the address is one tuple argument.

10 `.append()` mutates the list in place. The extra indent puts it inside the `if`, not merely inside the loop.

11 Closing inside the loop prevents file-descriptor exhaustion. Dedented one level, so it runs every iteration regardless of the `if`.

13 f-string interpolation. Plain `+` between a string and an int raises TypeError — a favourite planted bug; the fix is `str(port)` or an f-string.

ANATOMY 3 — POWERSHELL PORT CHECK

```
01 $target = "10.0.0.5"
02 $open   = @()
03
04 foreach ($port in 1..1024) {
05     $c = New-Object System.Net.Sockets.TcpClient
06     if ($c.ConnectAsync($target, $port).Wait(500)) {
07         $open += $port
08     }
09     $c.Close()
10 }
11 Write-Output "Open: $($open -join ', ')"
```

01 `$` prefixes every variable. Spaces around `=` are fine here — the opposite of Bash.

02 `@()` is an empty array literal.

04 `1..1024` is the range operator and is **inclusive at both ends** — unlike Python's `range()`. Braces delimit the block; indentation is cosmetic.

05 `New-Object` instantiates a .NET class. The `System.` namespace, and the bracket-and-double-colon form such as `[System.Net.Dns]::GetHostAddresses()`, are strong PowerShell tells.

06 `.Wait(500)` returns `$true` if the connection completed within 500 ms — this is the timeout. Comparison here would use `-eq` / `-gt`; `>` is *redirection*, not comparison.

07 `+=` on an array builds a new array each pass — acceptable at this scale, slow at large scale.

11 `$( )` is a subexpression evaluated inside the string; `-join` concatenates array elements with a separator.

- EXPECT If a loop body uses a hardcoded value where `$_` or `$port` belongs, every iteration tests the same target and produces 1024 identical results. Read the loop variable usage before anything else.

OPERATOR ANATOMY ACROSS THE THREE LANGUAGES

ELEMENT	BASH	PYTHON	POWERSHELL
Assignment	`x=5` (no spaces)	`x = 5`	`$x = 5`
Numeric equal	`[ "$a" -eq "$b" ]`	`a == b`	`$a -eq $b`
String equal	`[ "$a" = "$b" ]`	`a == b`	`$a -eq $b`
Greater than	`-gt`	`>`	`-gt` — never `>`
AND / OR / NOT	`&&` `||` `!`	`and` `or` `not`	`-and` `-or` `-not`
Regex match	`=~` in `[[ ]]`	`re.search()`	`-match`
Wildcard match	`==` in `[[ ]]`	`fnmatch`	`-like "*adm*"`
Loop over a range	`for i in $(seq 1 254)`	`for i in range(1,255)`	`foreach ($i in 1..254)`
Range inclusivity	both ends	**stop excluded**	both ends
Block delimiter	`do` … `done`, `then` … `fi`	indentation	`{ }`
Concatenate	`"$a$b"`	`a + b` (cast ints)	`"$a$b"` / `-join`
Substring	`${v:0:3}`	`v[0:3]`	`$v.Substring(0,3)`
Length	`${#v}`	`len(v)`	`$v.Length`
Arithmetic	`$(( a + b ))`	`a + b`	`$a + $b`
Exit status	`$?` — 0 = success	exceptions	`$?` / `$LASTEXITCODE`
Function	`name() { ... }`	`def name(a):`	`function Name { param($a) }`
Arguments	`$1 $2 $@`	`sys.argv`	`param()` / `$args`

DATA MANIPULATION — PARSING TOOL OUTPUT

- **Bash**
  TOOLS `grep` filter lines · `awk '{print $2}'` column · `cut -d: -f1` delimited field · `sed 's/a/b/'` substitute · `sort -u` · `uniq -c` · `tr` · `wc -l`
  EXPECT `cut` splits on a *single* character, so runs of spaces break it. Column extraction from aligned nmap or netstat output is `awk`; `cut` is the distractor.
- **Python**
  TOOLS `.split()` · `.strip()` · `.replace()` · list comprehension `[l for l in lines if "open" in l]` · `re` · `json` · `csv`
  EXPECT `with open(f) as fh:` closes the handle automatically; iterating a file object yields lines one at a time rather than loading the whole file.
- **PowerShell**
  TOOLS `Select-String` · `Where-Object { $_ -match "open" }` · `ForEach-Object` · `-split` · `Select-Object -First` · `Sort-Object -Unique` · `Export-Csv` · `ConvertFrom-Json`
  EXPECT PowerShell pipes *objects*, not text — which is why you filter on properties rather than pattern-matching a rendered string. This distinction is directly testable.

THE DEFECTS COMPTIA PLANTS — RANKED

DEFECT	WRONG	RIGHT
Off-by-one range	`range(1,1024)` for 1024 ports	`range(1,1025)`
Comparison operator	`if ($a > $b)` in PowerShell	`if ($a -gt $b)`
Loop variable unused	hardcoded port in the body	use `$_` / `$i` / `$port`
Unquoted variable	`[ $x -eq 5 ]` with `$x` empty	`[ "$x" -eq 5 ]`
No timeout	socket with OS default	`s.settimeout(0.5)`
Output truncated	`> results.txt` inside loop	`>>`, or `>` after `done`
Indentation	`print` dedented out of the loop	indent to match the block
Bash assignment	`x = 5`	`x=5`
Type mismatch	`"Port " + port` in Python	`f"Port {port}"`
Wrong connect call	`connect()` with no try/except	`connect_ex()` — 0 = open

- EXPECT Work in this order on every code question: identify the language → read the loop bounds → read the comparison operators → trace exactly one iteration by hand → check where output goes. That sequence finds most planted defects without reading the whole script.

2.4 TOOLS — GIVEN A SCENARIO, USE THE APPROPRIATE

TOOL	MODE	WHAT IT RETURNS	WHAT TO EXPECT ON THE EXAM
WHOIS	Passive	Domain registrar, registrant, dates, name servers; IP netblock owner, CIDR, ASN, abuse contact	"Who owns this range" / "confirm the client owns these IPs" — doubles as 1.1 scope validation
nslookup	Passive	DNS records, interactive mode	Chosen when the scenario specifies a Windows host with nothing installed
dig	Passive	Full DNS response with all sections	Chosen when output detail or scripting is emphasised
DNSdumpster	Passive	Subdomains, MX/host records, netblocks, visual map	"Quick passive map in a browser, nothing to install"
Amass	Both	Comprehensive subdomain enumeration, ASN/netblock discovery, graph output	"Most thorough" / "combine dozens of sources". Active mode adds brute force and zone transfers
Shodan	Passive	Banner-indexed internet devices; `port:` `product:` `org:` `vuln:`	Exposed ICS, SCADA, IoT, webcams, databases
Censys.io	Passive	Host attributes and deep certificate search	Certificate pivoting — finding the origin server behind a CDN
Wayback Machine	Passive	Historical site snapshots, retired pages and subdomains, old JS endpoints	Any wording about content "removed" or "no longer published"
Maltego	Passive	Entity graph built by transforms: domain → DNS → IP → netblock → person → email	"Visualise relationships" / "link analysis"
Recon-ng	Passive	Modular CLI framework, workspaces, module marketplace, database-backed results	"Metasploit-style module interface" / "workspaces"
SpiderFoot	Passive	Automated scan across hundreds of modules, correlated in a web UI	"Automated collection with minimal interaction"
OSINTframework.com	—	A categorised *directory* of OSINT resources	The trap: it collects nothing. Any question describing data collection has a different answer
theHarvester	Passive	Emails, subdomains, hosts, names, sometimes ports and banners, from many sources	"Command-line collection of emails and subdomains". Reappears in 4.8
Hunter.io	Passive	Email addresses, the org's naming pattern, deliverability verification	"Determine the email format" / "verify these addresses"
Nmap	**Active**	Host discovery, TCP/SYN/UDP scans, `-sV` versions, `-O` OS, `-T0..T5`, `-oA` output	The only unambiguously active scanner in this objective
Nmap NSE	**Active**	Lua scripts: `smb-enum-shares`, `http-enum`, `dns-brute`, `ssl-enum-ciphers`, `--script=vuln`	"Extend nmap to enumerate X". `intrusive` / `exploit` categories can crash services → an RoE question from 1.1
tcpdump	Passive	CLI capture with BPF filters, writes `.pcap`	"Capture on a headless server over SSH"
Wireshark	Passive	GUI dissection, follow stream, statistics, expert info; `tshark` for CLI	"Analyse the capture / follow the TCP stream"
WiGLE.net	Passive	Crowdsourced global wireless network database with geolocation	"Identify networks at a location you have not visited"
InSSIDer	Passive	Local survey: SSIDs, BSSIDs, channels, overlap, signal strength, encryption	"Assess channel overlap and coverage during a site survey"
Aircrack-ng	**Active**	airmon-ng monitor mode · airodump-ng capture · aireplay-ng injection and deauth · aircrack-ng WEP/WPA cracking	"Capture and crack the WPA2 handshake" / "deauthenticate a client". The only wireless tool that transmits

2.4 DIFFERENTIATORS

Shodan vs Censys — banner- and device-oriented (best ICS/IoT coverage) vs certificate- and dataset-oriented (best for cert pivoting to an origin server).
Maltego vs Recon-ng vs SpiderFoot — *Maltego visualises, Recon-ng modularises, SpiderFoot automates.*
theHarvester vs Hunter.io — free CLI generalist returning several data types vs commercial email specialist with format detection and verification.
Amass vs DNSdumpster — comprehensive CLI with passive *and* active modes vs quick passive browser lookup.
nslookup vs dig — available everywhere including Windows, simpler output vs richer, scriptable, Unix-standard.
tcpdump vs Wireshark — tcpdump *captures* (often headless); Wireshark *analyses* (often the file tcpdump produced).
WiGLE vs InSSIDer vs Aircrack-ng — a remote database you query vs a local passive survey vs the active attack suite.
OSINT Framework vs everything else — a directory of tools, not a tool that gathers data.